Foundations of Artificial Intelligence

KINTSUGI: AI POWERED INFRASTRUCTURE AUTOMATION

A PROJECT REPORT

Submitted by

2116240701056 – ASHVANTHAN SR

2116240701008 – ABINAV TL



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "AI POWERED INFRASTRUCTURE AUTOMATION: KINTSUGI "is the bonafide work of "Ashvanthan SR, Abinav TL" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**1**
1	**INTRODUCTION**	**2**
	1.1 BACKGROUND	2
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 SCOPE OF THE PROJECT	3
	1.5 ORGANISATION OF THE REPORT	4
2	**LITERATURE REVIEW**	**5**
	2.1 INTELLIGENT AGENTS	5
	2.2 LARGE LANGUAGE MODELS AND INSTRUCTION FOLLOWING	5
	2.3 TOOL-USING LANGUAGE MODELS	5
	2.4 RETRIEVAL AND GROUNDING	6
	2.5 SECURITY OF LLM-INTEGRATED APPLICATIONS	6
	2.6 HUMAN-AI INTERACTION	6
	2.7 INFRASTRUCTURE AS CODE	7
	2.8 SUMMARY OF GAPS	7
3	**PROPOSED SYSTEM**	**8**
	3.1 ARCHITECTURE DIAGRAM	8
	3.2 DESIGN PRINCIPLES	9
	3.3 THE AGENT IN PEAS TERMS	10
	3.4 REQUEST PROCESSING FLOW	10
	3.5 SCRIPT LIFECYCLE	11
	3.6 TECHNOLOGY STACK	12
4	**MODULES DESCRIPTION**	**13**
	4.1 LLM AGENT AND TOOL ORCHESTRATOR	13
	4.2 RULE-BASED FALLBACK ROUTER	14
	4.3 CREDENTIAL VAULT AND CONNECTION MANAGER	15
	4.4 SCHEDULE ANALYSIS AND CRON ENGINE	16
	4.5 SCRIPT ENGINE, POLICY CHECKS AND HUMAN APPROVAL	17
	4.6 TASK TEMPLATES, CLARIFYING QUESTIONS AND PREREQUISITES	17
	4.7 AUDIT LOGGING	18
	4.8 MULTI-CLOUD CONNECTORS	18
	4.9 DESKTOP USER INTERFACE	19
5	**IMPLEMENTATION AND RESULTS**	**20**

5.1 EXPERIMENTAL SETUP 20
5.2 RESULTS 20
5.3 END-TO-END CASE STUDY: PUBLIC STATIC WEBSITE ON AMAZON S3 23
6 CONCLUSION AND FUTURE WORK 30
6.1 CONCLUSION 30
6.2 FUTURE WORK 30
7 REFERENCES 32

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Architecture diagram of Kintsugi	9
3.2	Life cycle of a write script	11
4.1	Connections page for adding a verified AWS connection	16
5.1	Chat page: request with an attached file, clarifying questions and the saved script	24
5.2	Scripts page listing scripts waiting for review	25
5.3	Script #36 before approval, with Run now and Schedule locked	26
5.4	Script #36 after approval: all eight steps completed	27
5.5	AWS console: the bucket created by Kintsugi	28
5.6	AWS console: versioned index.html uploaded to the bucket root	28
5.7	Audit log with every step of the run recorded	29

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.1	PEAS description of the Kintsugi agent	10
3.2	Technology stack	12
4.1	Main tools available to the agent	14
5.1	Hardware and software environment	20
5.2	Unit test results	21
5.3	Safety test cases	22

ABSTRACT

Managing cloud infrastructure on cloud providers such as Amazon Web Services (AWS) requires knowledge of hundreds of services, consoles and command-line options, and a single mistake can expose data or create unexpected bills. This project presents Kintsugi, a native desktop application that lets a user manage an AWS account in plain English through an intelligent agent built on a locally hosted large language model (Qwen3-14B served by LM Studio). The agent follows the tool-calling paradigm: the language model only chooses typed tools and explains results, while ordinary Python code calls AWS, performs every calculation (cron schedules, time zones, costs) and enforces safety rules. Read requests are answered immediately; every change is converted into a reviewable script that runs only after an explicit human Approve click, with typed confirmation for destructive actions. A rule-based fallback router, clarifying-question templates, a prerequisite checker and more than twenty-five deployment recipes (EC2, S3, Lambda, RDS, ECS, EKS, VPC, Terraform, Ansible) make the agent dependable even with a small model. Credentials are encrypted with Fernet, verified with AWS STS and never shown to the model, and every action is audited in PostgreSQL. The system is covered by 274 automated unit tests, all passing. An end-to-end run in which a single chat request produced a versioned, public static website on Amazon S3, verified by an automatic HTTP check and recorded in the audit log, demonstrates the complete workflow.

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Cloud computing has become the default platform for hosting applications, databases and data pipelines. Amazon Web Services alone offers more than two hundred services, each with its own console pages, API operations and configuration options. Engineers who manage these services must remember exact command syntax, the order in which resources depend on one another (for example, a database needs a subnet group, which needs subnets in two availability zones, which need a VPC), and the security implications of every setting. For students and small teams this complexity is a real barrier, and for experienced engineers it is a frequent source of costly mistakes.

At the same time, large language models (LLMs) have shown that natural language can be an effective interface to complex systems. Modern models can be connected to external tools through function calling: instead of answering only from memory, the model emits a structured request such as list_ec2_instances(state='running'), the host program executes it, and the model explains the result. This turns the LLM into the reasoning component of an intelligent agent that perceives its environment through tools and acts on it through tools.

1.2 PROBLEM STATEMENT

Giving an AI model direct access to a cloud account is dangerous. Language models can hallucinate resource identifiers, make arithmetic mistakes with times and costs, misunderstand an ambiguous request or be manipulated by text inside the data they read. A cloud assistant therefore has to answer three questions at once: how can a user describe a task in plain language, how can the

system complete that task across many AWS services, and how can it guarantee that nothing harmful or unintended happens without the user's explicit consent?

1.3 OBJECTIVES

- Build a native desktop application (no browser) in which a user manages AWS through a chat interface.
- Use a local, free and offline LLM (Qwen3-14B through LM Studio) so that no data leaves the machine.
- Restrict the model to choosing typed tools; all AWS calls, calculations and safety checks are performed by code.
- Answer read questions directly, and turn every write request into a reviewable script that runs only after human approval.
- Ask clarifying questions and check prerequisites before building anything, instead of letting the model guess.
- Store credentials securely (encrypted, verified, never shown to the model) and audit every action.
- Support multi-step goals such as hosting a website, running a container on ECS Fargate or creating a database through reusable recipes.

1.4 SCOPE OF THE PROJECT

The application targets AWS as the fully featured provider, with basic connectors for Microsoft Azure and Google Cloud (list and start/stop virtual machines, list storage, upload files). It manages cloud resources only; it does not connect to the user's own databases. It runs entirely on the user's computer as a single Python process, with a local PostgreSQL database that stores application state (conversations, scripts, audit records and encrypted credentials). Tasks that AWS itself requires a human for, such as account creation, MFA setup, support cases, Marketplace subscriptions and service quota increases, are explicitly out of scope and the application says so when asked.

1.5 ORGANISATION OF THE REPORT

Chapter 2 reviews related work on tool-using language models, intelligent agents and cloud automation. Chapter 3 presents the proposed system and its architecture. Chapter 4 describes each module in detail. Chapter 5 covers the experimental setup and results, and Chapter 6 concludes the report with future work.

CHAPTER 2
LITERATURE REVIEW

2.1 INTELLIGENT AGENTS

Russell and Norvig (2021) define an intelligent agent as anything that perceives its environment through sensors and acts upon it through actuators, and they describe agents through the PEAS framework (Performance measure, Environment, Actuators, Sensors). A goal-based agent chooses actions that move the environment towards a desired state, and a utility-based agent additionally weighs outcomes such as cost and risk. Kintsugi follows this model directly: AWS read APIs are its sensors, approved AWS write calls are its actuators, and the user's request defines the goal.

2.2 LARGE LANGUAGE MODELS AND INSTRUCTION FOLLOWING

Brown et al. (2020) showed that large transformer language models can perform new tasks from a few examples in the prompt. Ouyang et al. (2022) improved instruction following by fine-tuning with human feedback, which made models usable as assistants. Wei et al. (2022) showed that intermediate reasoning steps improve accuracy on multi-step problems. Recent open-weight models such as Qwen3 (Qwen Team, 2025) bring these capabilities, including native function calling, to models small enough to run on a personal computer, which makes a fully local cloud assistant possible.

2.3 TOOL-USING LANGUAGE MODELS

Yao et al. (2023) proposed ReAct, in which a model interleaves reasoning with actions such as search queries and observes the results before continuing. Schick et al. (2023) introduced Toolformer, a model that learns when to call external APIs such as a calculator, motivated by the observation that language models are weak at exact arithmetic. Patil et al. (2023) presented Gorilla, which

connects an LLM to a large set of APIs and reports that models frequently hallucinate API names and arguments unless they are grounded in documentation. These findings shape this project: the model never computes numbers itself, and every tool call is validated against a schema before it is executed.

2.4 RETRIEVAL AND GROUNDING

Lewis et al. (2020) introduced retrieval-augmented generation (RAG), which grounds a model's answer in documents retrieved at query time and reduces hallucination. Kintsugi applies the same idea to live infrastructure: answers are grounded in the data returned by AWS read calls at the time of the question, and the options offered to the user (availability zones, subnets, instance classes) are looked up live rather than generated by the model.

2.5 SECURITY OF LLM-INTEGRATED APPLICATIONS

Saltzer and Schroeder (1975) formulated the principle of least privilege: every program should operate with the smallest set of privileges needed for its job. Greshake et al. (2023) demonstrated indirect prompt injection, in which instructions hidden in data that an LLM reads can take control of an LLM-integrated application. Together these works argue that the model must never be the security boundary. In Kintsugi, AWS Identity and Access Management (IAM) remains the hard boundary, and the application adds policy checks and human approval on top of it.

2.6 HUMAN-AI INTERACTION

Amershi et al. (2019) proposed guidelines for human-AI interaction, including making clear what the system can do, supporting efficient correction and dismissal, and asking for clarification when uncertain. The approval workflow (Approve, Deny, Edit), the clarifying-question forms and the explicit statement of limits in Kintsugi follow these guidelines.

2.7 INFRASTRUCTURE AS CODE

Brikman (2022) describes infrastructure as code, in which infrastructure is defined in reviewable, versioned files and changes are previewed with a plan before they are applied. Tools such as Terraform and Ansible make changes repeatable and auditable. Kintsugi adopts the same plan-review-apply pattern: every write request becomes a script that can be read, dry-run, edited and approved before execution, and every script carries an automatically generated teardown.

2.8 SUMMARY OF GAPS

Existing cloud consoles and command-line tools are powerful but require expert knowledge. Existing LLM assistants either only answer documentation questions or run commands with little control. There is a gap for an assistant that combines natural-language interaction with strict, code-enforced safety, runs locally without paid APIs, and remains reliable with a small open model. This project addresses that gap.

CHAPTER 3
PROPOSED SYSTEM

Kintsugi is a native desktop application written in Python with a PySide6 (Qt) user interface. The user types a request in the chat window, for example "What Lambda functions are scheduled to run after 10am?" or "Host the folder I attached as a website on S3". A local LLM interprets the request and selects tools; Python code executes those tools against AWS and returns clean, flattened data; the model then explains the result in plain language. Requests that change the account never run directly: they are saved as scripts that the user reviews and approves on the Scripts page.

3.1 ARCHITECTURE DIAGRAM

Figure 3.1 shows the layered architecture. The GUI sends the request to the agent loop, which runs in a background QThread. The agent exchanges messages with the local LLM through an OpenAI-compatible API. Tool calls pass through the Tool Orchestrator, which contains the tool registry, the task templates and recipes, the prerequisite checker and the policy-checked script engine. Write operations wait for human approval; when they run, the credential vault supplies a session for the required purpose (read or write) and the step is executed against AWS or a local tool. Every action is written to the audit log in PostgreSQL.

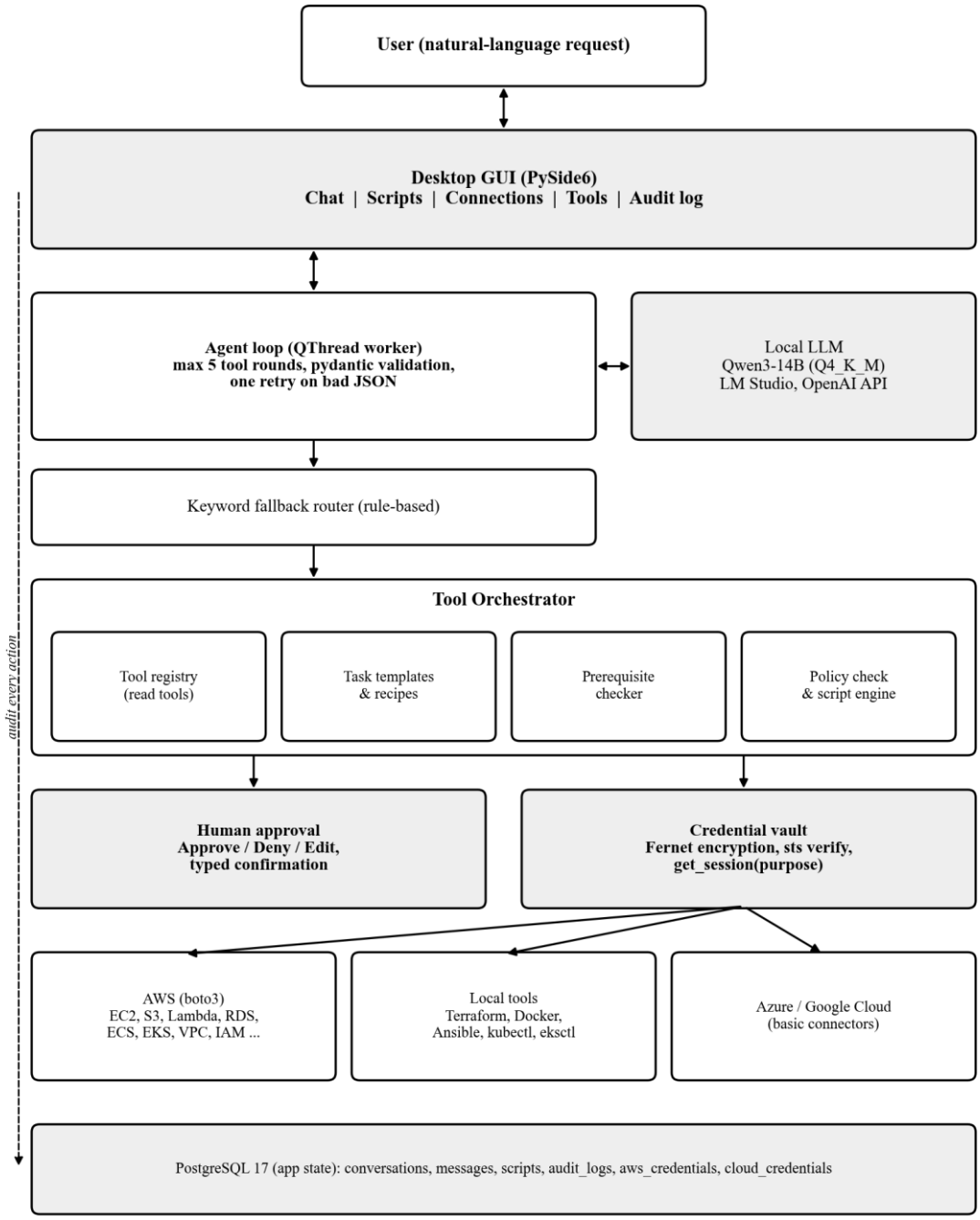


Fig 3.1 Architecture diagram of Kintsugi

3.2 DESIGN PRINCIPLES

1. **IAM is the real security boundary.** Application checks are a second layer, never a replacement.
2. **The LLM never calls AWS directly and never sees credentials.** It only chooses from typed tools and recipes.

3. **Numbers are computed by code.** Cost, utilisation, cron times and time-zone conversions are never left to the model.
4. **Reads are free; writes need approval.** Destructive actions additionally need a typed confirmation.
5. **Secrets are never logged** or sent to the model, the audit table or error messages.
6. **Core logic is independent of the GUI.** Slow work (LLM, AWS, Terraform, Docker) runs in background threads.

3.3 THE AGENT IN PEAS TERMS

Table 3.1 PEAS description of the Kintsugi agent

Component	Description
Performance measure	Correct answers, correct tool choice, tasks completed, zero unapproved changes, zero leaked secrets
Environment	The user's AWS account (and basic Azure/GCP), the local file system, the chat conversation
Actuators	Approved AWS API calls, Terraform, Docker, Ansible, kubectl commands, chat replies
Sensors	AWS read APIs (describe/list/get), user messages, attachments (paths only), prerequisite lookups

The environment is partially observable (the agent sees only what read APIs return), stochastic (resources may fail to create), sequential (later steps depend on earlier outputs) and dynamic (other users can change the account). These properties justify the design choices of live lookups, waiters, per-step undo actions and a human in the loop.

3.4 REQUEST PROCESSING FLOW

1. The user sends a message, optionally with attached files or folders (only their paths are given to the model).
2. The agent sends the conversation and the tool schemas to the LLM with temperature 0.2.

3. If the model returns a tool call, its arguments are validated with pydantic; on invalid JSON or arguments the model gets one chance to correct them.
4. If the model calls no tool, calls a clearly wrong tool, or the LLM server is unavailable, the keyword fallback router selects the tool instead.
5. Read tools run immediately with the read-only identity and return clean, truncated data (about 6000 characters).
6. Build requests go to plan_task: the matching template checks prerequisites, asks clarifying questions, and produces a script.
7. The script is saved as a draft. The user approves, denies or edits it; only approved scripts can run or be scheduled.
8. At most five tool rounds are allowed per request; the model then writes the final answer from the tool results.

3.5 SCRIPT LIFECYCLE

Every change to the cloud account is represented as a script, which is a list of typed steps. Figure 3.2 shows the life cycle of a script. The run and schedule operations check the status in the core layer, not only in the GUI, so a draft or denied script can never be executed.

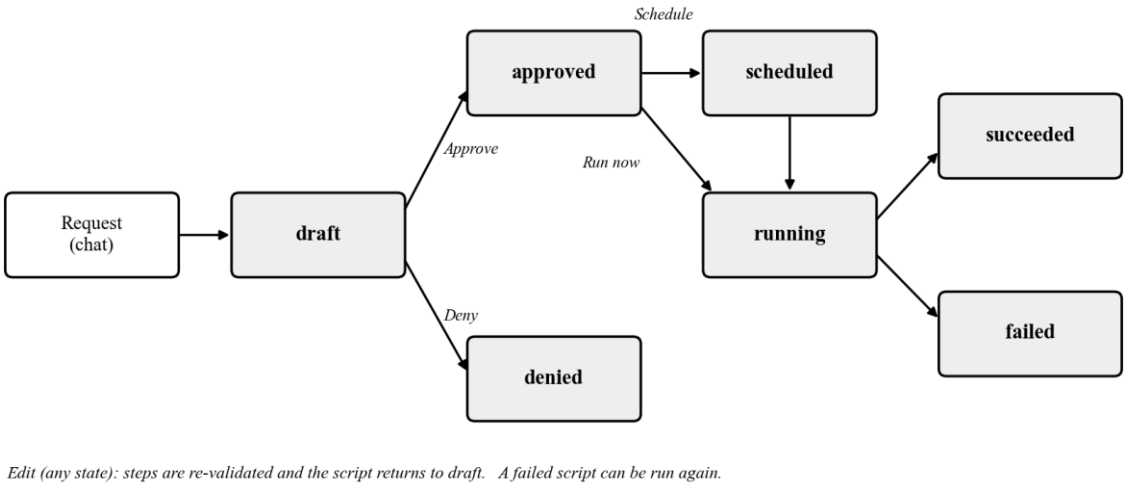


Fig 3.2 Life cycle of a write script

3.6 TECHNOLOGY STACK

Table 3.2 Technology stack

Layer	Technology
User interface	PySide6 (Qt 6), Fusion theme, QThread workers
Language model	Qwen3-14B (GGUF Q4_K_M) in LM Studio, OpenAI-compatible API, temperature 0.2
Cloud access	boto3 / botocore (AWS), REST APIs for Azure and Google Cloud, Azure CLI
Validation	pydantic schemas, botocore service models
Security	cryptography (Fernet), AWS STS verification, IAM least privilege
Database	PostgreSQL 17 with psycopg2
Automation	Terraform, Docker, Ansible (in a Docker image), eksctl, kubectl, Helm
Testing	Python unittest with fake AWS clients (274 tests)

CHAPTER 4
MODULES DESCRIPTION

The source code is organised as a modular monolith. The core package contains plain Python modules that can be tested without a user interface; the gui package contains only presentation code and calls core functions. The main modules are described below.

4.1 LLM AGENT AND TOOL ORCHESTRATOR

The agent module (core/agent.py) implements the perceive-reason-act loop. It sends a system prompt that states the rules (always use tools for facts, never invent identifiers, never do time or cost arithmetic, always state the time zone, never claim a change was made) together with the JSON schemas of all tools. The LLM client (core/llm.py) appends /no_think to disable the model's long reasoning mode for speed and strips any <think> blocks from the output.

Each tool is declared with a name, a description, a pydantic input schema and a risk level (read, operate or deploy). Before a tool runs, its arguments are validated; invalid arguments are returned to the model with the error message so it can correct them once. The number of tool rounds is limited to five per request to prevent loops. The agent also detects when a request starts a new task and drops earlier conversation so that names and settings from a finished task are not reused by mistake. Table 4.1 lists the main tools available to the agent.

Table 4.1 Main tools available to the agent

Tool	Purpose	Risk
aws_read	Generic read of any AWS service (list_, describe_, get_ only); secret-returning operations are blocked	read
list_ec2_instances, list_lambda_functions, list_s3_buckets, list_rds_instances, list_load_balancers	Curated listings with clean, flattened output	read
get_ec2_cpu, get_cost_by_service	CloudWatch CPU and Cost Explorer figures, computed by code	read
list_scheduled_lambdas	EventBridge rules and Scheduler schedules, next runs in the user's time zone	read
list_s3_objects, download_s3_object, download_lambda_code	Browse and download; file contents never go to the LLM	read
check_prerequisites	Prerequisite checklist for any AWS service	read
list_cloud_vms, list_cloud_storage	Azure and Google Cloud virtual machines and storage	read
plan_task	Select a recipe, ask clarifying questions, produce a script	operate
aws_write	Any single AWS write call, saved as a draft script	operate
stop_instance, start_instance	Stop or start an EC2 instance after Approve	operate
cloud_vm_action, cloud_upload	Azure / Google Cloud VM start/stop and file upload, as scripts	operate

4.2 RULE-BASED FALLBACK ROUTER

A 14-billion-parameter model occasionally answers without calling a tool or chooses a related but wrong tool. The router (core/router.py) is a small rule-based expert system made of production rules over keywords and patterns. For example, a message containing "stop" together with an instance identifier or name maps to stop_instance; "schedule", "cron", "runs at" or "every" together with Lambda maps to list_scheduled_lambdas; words such as create, deploy or host together with a service name map to the matching recipe through plan_task, and services without a recipe map to check_prerequisites. Extraction functions pull out instance names, function names, bucket names, file paths and times from the text. The router is used when the model calls no tool, calls a known-wrong tool,

or when LM Studio is not running, so the application can still answer basic questions offline.

4.3 CREDENTIAL VAULT AND CONNECTION MANAGER

Users add their own cloud connections on the Connections page, in a form modelled on the aws configure command. Supported modes are AssumeRole (role ARN and external ID), a local AWS CLI profile, and access keys. Before a connection is saved, it is verified with sts get-caller-identity, and root credentials are rejected. The secret key is encrypted with Fernet (AES in CBC mode with an HMAC) using a key held in the local .env file, only the last four characters are shown afterwards, and the input field is cleared after saving. The function vault.get_session(owner, purpose) is the only place where secrets are decrypted; a read purpose may fall back to the configured read-only profile, but write purposes never fall back. Azure (service principal or Azure CLI login) and Google Cloud (service-account key) connections are stored in the same encrypted way.

Figure 4.1 shows the Connections page. The user chooses the cloud (AWS, Azure or Google Cloud), enters a connection name, the purpose of the connection (Read for answering questions in chat, Write for running approved scripts), the access key ID, the masked secret access key and the region. Test connection calls sts get-caller-identity; Save connection encrypts the secret and stores it only after the verification succeeds. The advanced option switches to a local AWS CLI profile or an IAM role.

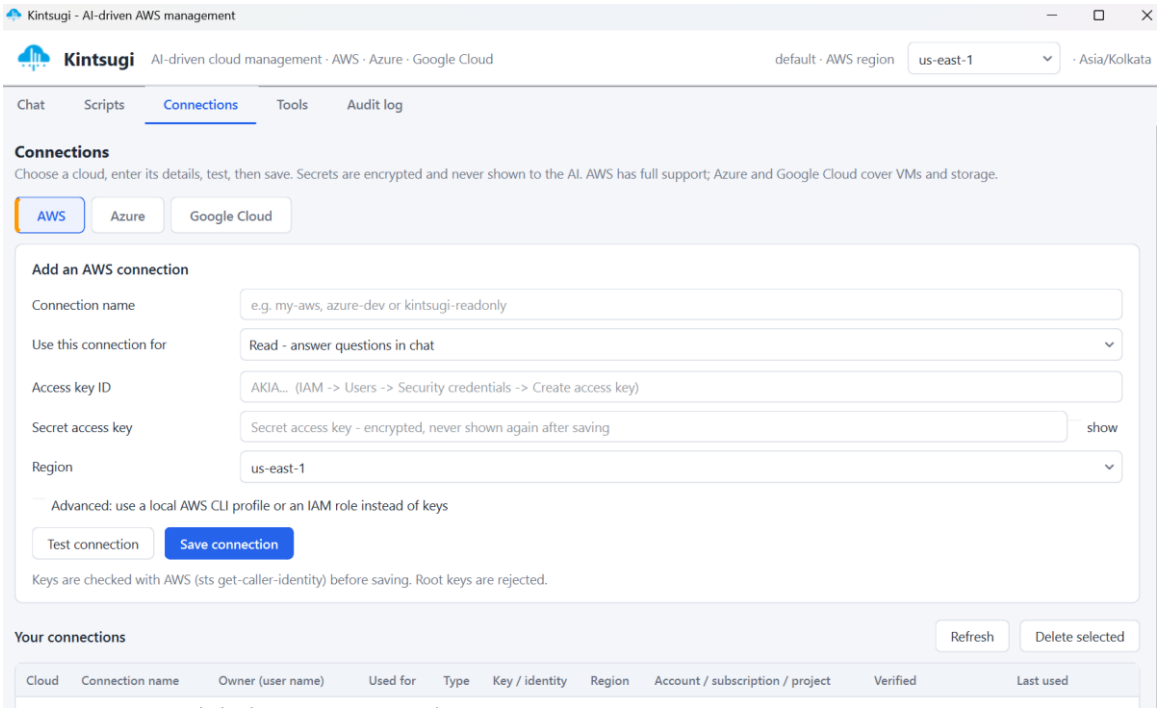


Fig 4.1 Connections page for adding a verified AWS connection

4.4 SCHEDULE ANALYSIS AND CRON ENGINE

The first acceptance test asks: "What Lambda functions are scheduled to run after 10am?" Lambda does not store schedules; they live in Amazon EventBridge rules and EventBridge Scheduler. The schedules module (core/schedules.py) pages through events.list_rules and list_targets_by_rule, and through scheduler.list_schedules and get_schedule, keeping only targets whose ARN refers to a Lambda function. The cron module (core/cron.py) parses AWS cron expressions, which have six fields (minute, hour, day-of-month, month, day-of-week, year), use ? and number days of the week from 1 (Sunday) to 7. It computes the next run times in code, converts them from UTC (for rules) or from the schedule's own time zone to the user's time zone (Asia/Kolkata), and filters runs after the requested time. rate() expressions have no clock time and are reported separately, and disabled schedules appear only as a note. The model only formats the resulting table, so it never performs date arithmetic.

4.5 SCRIPT ENGINE, POLICY CHECKS AND HUMAN APPROVAL

The script engine (core/scripts.py) converts every write into a script made of typed steps. Each AWS step is checked against the botocore service model, so unknown operations or parameters are rejected with suggestions, and against a write policy. The policy blocks Organizations, account and STS writes, the creation of access keys and login passwords, AdministratorAccess and wildcard IAM policies, public AMI or snapshot sharing, SSH (port 22) open to 0.0.0.0/0, and literal passwords inside scripts (databases use passwords generated into AWS Secrets Manager instead). Fake example identifiers such as vpc-12345678 are rejected. IAM changes, public S3 websites and other risky but legitimate actions are allowed only with a typed confirmation of the form "run script <id>".

Each script is rendered as equivalent AWS CLI commands so that the user can read exactly what will happen. Steps may reference outputs of earlier steps through placeholders, wait for resources with waiters, and declare an undo action. After a run, even a partially failed one, an automatic teardown script is generated from the undo actions. Scripts can be dry-run, edited as JSON (which re-validates everything and returns the script to draft), scheduled in the user's time zone, or run immediately.

4.6 TASK TEMPLATES, CLARIFYING QUESTIONS AND PREREQUISITES

Building infrastructure requires many decisions, and a small model tends to invent the missing ones. Task templates (core/templates.py) describe each task as a set of fields with types, defaults, allowed options and a must_ask flag. This is a frame-based dialogue approach: the template is a frame and the fields are slots. When a request arrives, code determines which slots are still empty and asks the user, both as a numbered list in the chat and as a form with live options (availability zones, subnets, security groups, key pairs, the user's public IP

address, instance classes actually orderable in the region). Values for must_ask fields, such as CIDR ranges, NAT gateways or SSH source addresses, are never accepted from a model guess until the user confirms them.

Before any question is asked, the prerequisite checker (core/prereqs.py) verifies, with read-only lookups, that everything the task depends on exists, for example a VPC with subnets in two zones for a database, a key pair for an EC2 instance, or Docker and Terraform installed locally. Each item is reported as ready, missing, created by the script, manual or unknown. A required missing item blocks the build and is shown as a checklist with a Check again button. For AWS services without a recipe, the checker reads the botocore model of the main create operation and checks the resources it references. This mirrors the precondition checking of classical AI planners.

4.8 AUDIT LOGGING

The audit module (core/audit.py) records every AI action in the audit_logs table: timestamp, actor, the original request, tool name, arguments, risk level, approval state, status and a result summary. Arguments are passed through a redaction function that removes keys, passwords, tokens and similar values before they are stored. Audit writes are wrapped so that a database failure can never crash the application; the user's action still completes and the failure is reported.

4.9 MULTI-CLOUD CONNECTORS

A common Connector interface (core/clouds.py) defines verify, list_vms, vm_action, list_storage and upload_file. AzureConnector uses Azure Resource Manager REST APIs with either a service principal or tokens from the Azure CLI, and GcpConnector signs a JSON Web Token locally from a service-account key. These connectors use only the Python standard library for HTTP, so no extra SDKs are required. AWS remains the fully featured provider.

4.10 DESKTOP USER INTERFACE

The GUI (gui/) has five tabs. Chat shows the conversation with question forms, prerequisite cards and file or folder attachments (drag and drop supported). Scripts lists every script with its status and offers Approve, Deny, Edit, Dry run, Run now, Schedule and Delete, with live output while a script runs. Connections manages AWS, Azure and Google Cloud credentials. Tools checks which local tools (Docker, Terraform, kubectl, eksctl, Helm, Azure CLI) are installed. Audit log shows the audit table. All slow work runs in QThread workers so the window never freezes. Screenshots of these pages, taken from a real run, are shown in Figure 4.1 and in Section 5.3.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

Table 5.1 Hardware and software environment

Item	Details
Operating system	Any OS supporting PYside6
Programming language	Python 3.10
Language model	Qwen3-14B, GGUF Q4_K_M (about 9 GB), partial GPU offload
Model server	LM Studio local server, http://localhost:1234/v1, context 16384 tokens
Model settings	Temperature 0.2, thinking disabled (/no_think), repeat penalty 1.0
Database	PostgreSQL 17 (application state only)
Cloud	AWS account with a dedicated read-only IAM user (ReadOnlyAccess + Cost Explorer read); separate write connection
Local tools	AWS CLI v2, Azure CLI 2.91, Terraform, Docker Desktop, eksctl, Helm

AWS resources for testing were created in a personal account protected by MFA on the root user and a budget alert of five US dollars per month. The application was always used with dedicated IAM users, never with root credentials. Unit tests use fake AWS clients, so they run offline, cost nothing and never modify a real account.

5.2 RESULTS

5.2.1 Automated testing

The project is covered by 274 automated unit tests written with Python's unittest framework. All 274 tests pass (run time about 13 seconds). Table 5.2 summarises the tests by area.

Table 5.2 Unit test results

Test file	Area tested	Tests	Result
test_cron.py	AWS cron/rate/at parsing, next runs, time zones	17	Pass
test_schedules.py	EventBridge rules and Scheduler to Lambda	9	Pass
test_tools.py	Read tools, blocklist, clean output	25	Pass
test_scripts.py	Script validation, policy, approval states	19	Pass
test_templates.py	Clarifying questions, must_ask fields	23	Pass
test_prereqs.py	Prerequisite checklist	16	Pass
test_recipes.py	Core recipes	28	Pass
test_infra_recipes.py	Network, storage, container, Lambda recipes	45	Pass
test_compute.py	Load balancer, ASG, AMI, EBS recipes	22	Pass
test_database_custom.py	Database recipe custom settings	14	Pass
test_s3_website.py	Public S3 static website	12	Pass
test_local_steps.py	Local commands, scrubbing, Ansible	14	Pass
test_multicloud_s3.py	Azure/GCP connectors, S3 upload	13	Pass
test_vault.py	Encryption, verification, root rejection	7	Pass
test_azure_cli.py	Azure CLI sign-in	6	Pass
test_audit.py	Audit redaction and failure safety	4	Pass
Total		274	All pass

5.2.2 Functional verification

The following behaviours were verified against the real database, the local LLM and the AWS account:

- Connecting to PostgreSQL and LM Studio; the agent answers read questions using the tools.
- A request to create an S3 bucket produces a draft script on the Scripts page and nothing runs until it is approved.
- A request to build a VPC triggers clarifying questions; after the answers are given in chat, a 20-step script with teardown is generated.
- A request for an EC2 instance asks eight questions with live options (zones, subnets, security groups, key pairs, the user's public IP).

- Read-only planning for every recipe works against the live account, and the prerequisite checker correctly blocks builds when the region has no VPC.
- Terraform plan, apply and teardown destroy run end to end through the script engine.
- The desktop GUI renders all question forms and pages with the real database.
- A public static website with bucket versioning was built on Amazon S3 from one chat request with an attached index.html; the eight-step script ran only after approval, the HTTP check succeeded and every step was audited (Section 5.3).

5.2.3 Safety evaluation

The policy layer was tested with requests that should be refused or require extra confirmation. Table 5.3 shows the expected and observed behaviour; each case is covered by a unit test.

Table 5.3 Safety test cases

Request or input	Expected behaviour	Observed
Write request from chat	Saved as draft script, not executed	As expected
Run a draft or denied script	Refused by the core layer	As expected
Security group with SSH from 0.0.0.0/0	Blocked	As expected
IAM policy with Action '*' on Resource '*'	Blocked	As expected
Create IAM access key	Blocked, user told to use the console	As expected
Database password written in the script	Rejected; AWS-managed password used	As expected
Placeholder ID such as vpc-12345678	Rejected at save time	As expected
Public S3 website	Allowed only with typed confirmation	As expected
Root access keys in Connections	Rejected after STS verification	As expected
Secret values in tool arguments	Redacted before audit storage	As expected

5.2.4 Observations

Moving all calculations into code proved essential: the cron and time-zone results are exact and tested, whereas the model on its own frequently miscalculates UTC to IST conversions. Clean, flattened tool output greatly improved the quality of the 14B model's answers compared with raw API responses. The model still occasionally produces a malformed tool call as plain text or chooses a related but wrong tool, especially on long multi-part requests; the argument validation with one retry and the rule-based fallback router recover most of these cases, and in the remaining cases the model reports the error instead of changing anything. Response time depends on GPU offload; with partial offload the 14B model typically needs several seconds per tool round. The case study in Section 5.3 shows the fallback path in practice: the LM Studio request failed with a BadRequestError, yet the keyword router still selected plan_task, the template asked its questions and a correct script was produced.

5.3 END-TO-END CASE STUDY: PUBLIC STATIC WEBSITE ON AMAZON S3

To validate the complete pipeline against a real AWS account, the following request was typed in the chat with the file webapptesting/site/index.html attached: "Build a static web application on the S3 bucket of the AWS and also enable bucket versioning and also allow static web hosting on the web s3 bucket". The screenshots below were taken during this run on 28 October 2026 in the us-east-1 region. They show each stage of the agent's perceive-reason-act cycle and the human-in-the-loop approval.

5.3.1 Request, tool selection and clarifying questions

Figure 5.1 shows the Chat page. At the time of this run the LLM call returned a BadRequestError, so the agent reported that the AI explanation was unavailable and the rule-based fallback router selected the plan_task tool from

the keywords "S3", "static" and "web hosting". The s3_static_website template then validated the extracted values: the word "versioning" had been taken as a bucket name, which is not one of the user's buckets, so the field was highlighted and the user was asked again (Which bucket? Enable bucket versioning?). The template also explained, in code-generated text, that the bucket will be public, that Block Public Access is turned off for this bucket only, and what it costs. After the answers were confirmed, script #36 was saved as destructive and waiting for Approve or Deny. Nothing had been changed in AWS at this point.

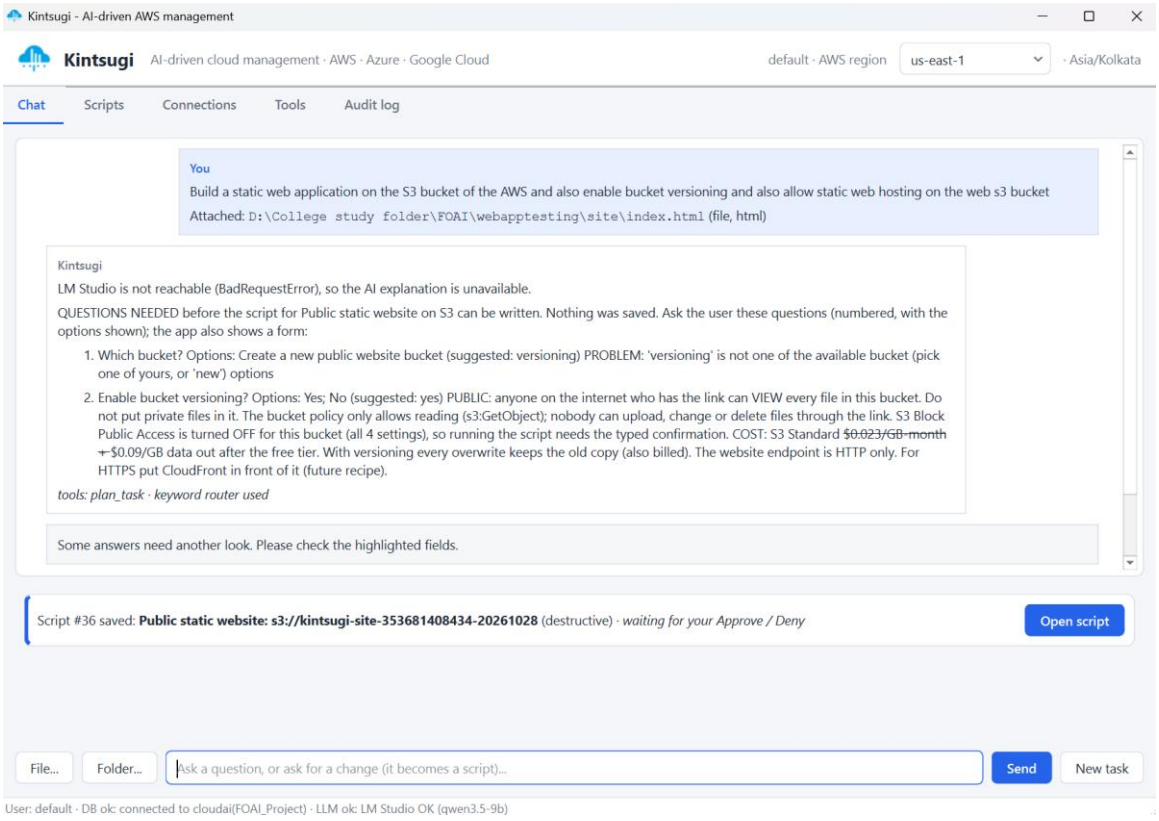


Fig 5.1 Chat page: request with an attached file, clarifying questions and the saved script

5.3.2 Script review and approval

The Scripts page (Figure 5.2) lists every script with its status, whether it is destructive, its schedule and creation time. Script #36 and an earlier VPC script #35 are both waiting for review, which shows that the agent never runs a change on its own.

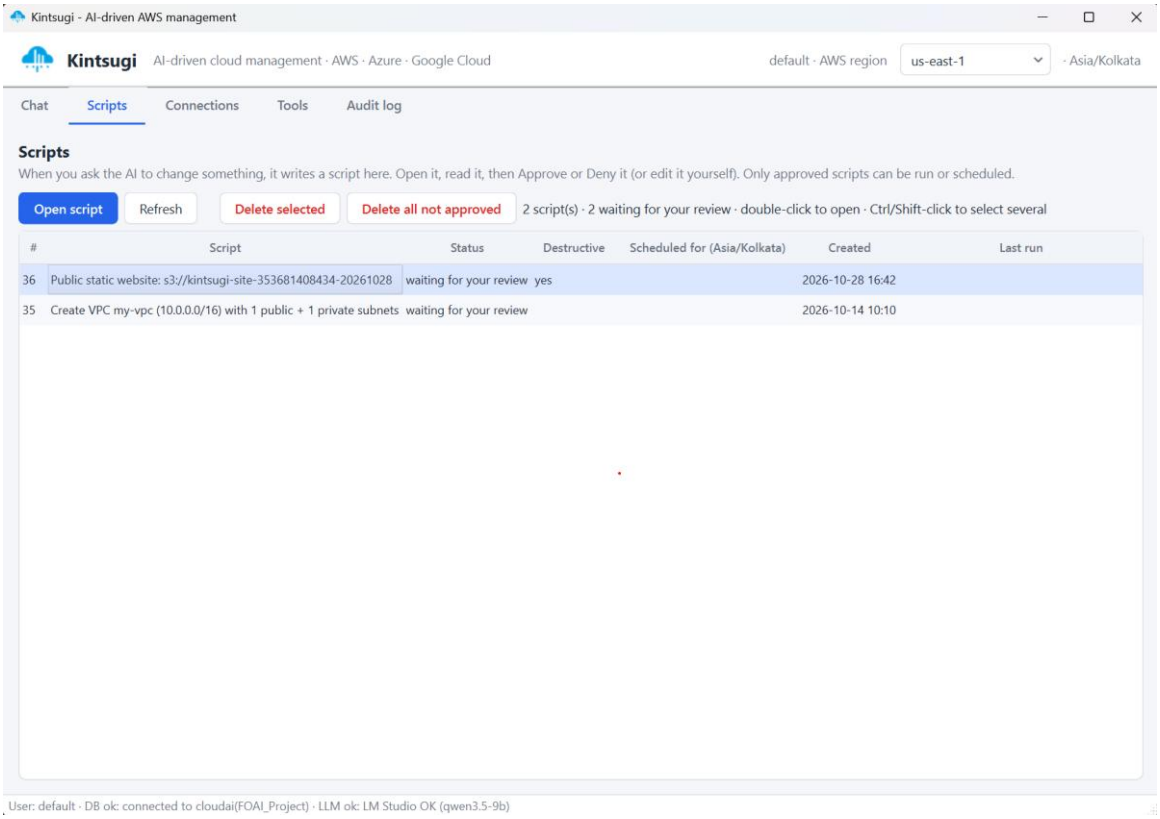


Fig 5.2 Scripts page listing scripts waiting for review

Opening the script (Figure 5.3) shows the equivalent AWS commands with notes for the reviewer: the website URL that will be created, the public-access warning, the cost note and the fact that the website endpoint is HTTP only. Because the script makes a bucket public, the header warns that a typed confirmation is required. Run now and Schedule stay disabled until the script is approved; only Dry run is available.

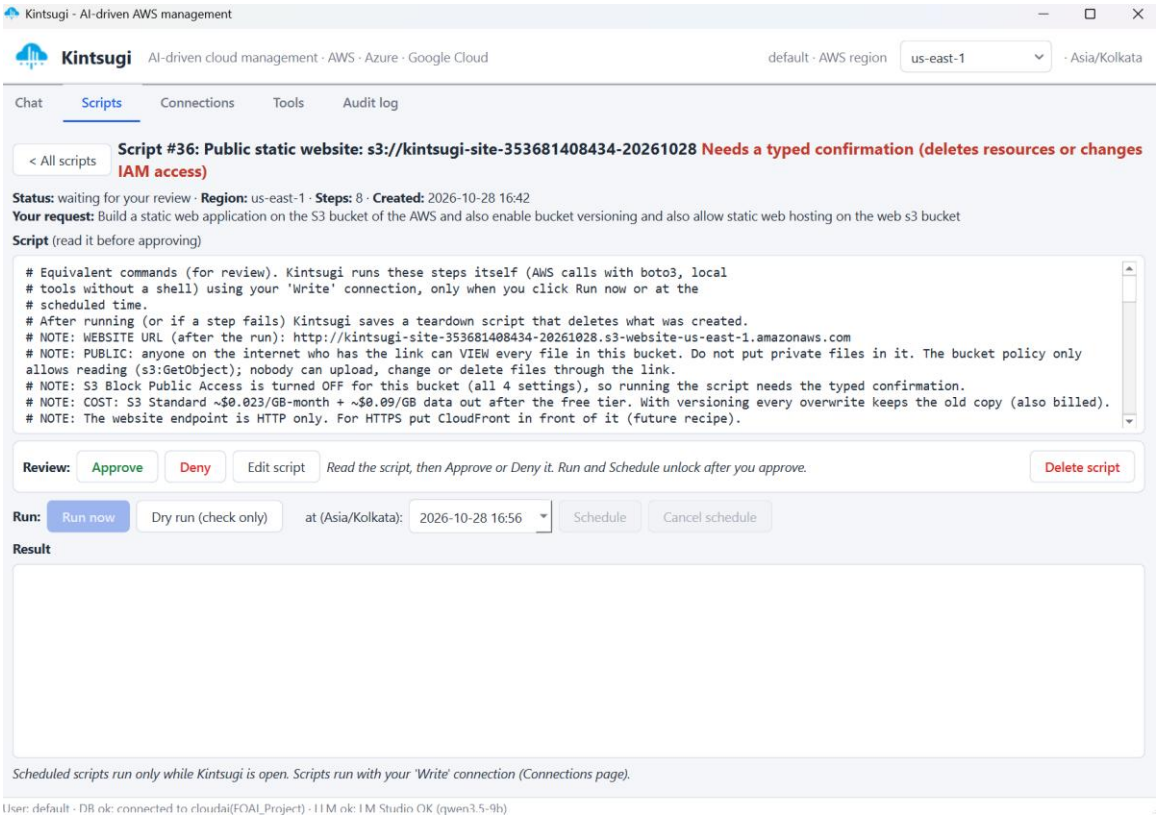


Fig 5.3 Script #36 before approval, with Run now and Schedule locked

5.3.3 Execution and verification

After the user clicked Approve and typed the confirmation, the script ran with the Write connection. Figure 5.4 shows the result: all eight steps finished with OK, namely create bucket, tag the bucket, enable versioning, turn off the four Block Public Access settings for this bucket, attach a bucket policy that allows only s3:GetObject, turn on static website hosting with index.html, upload the attached file to the root of the bucket, and finally an HTTP check of the website URL, which answered successfully.

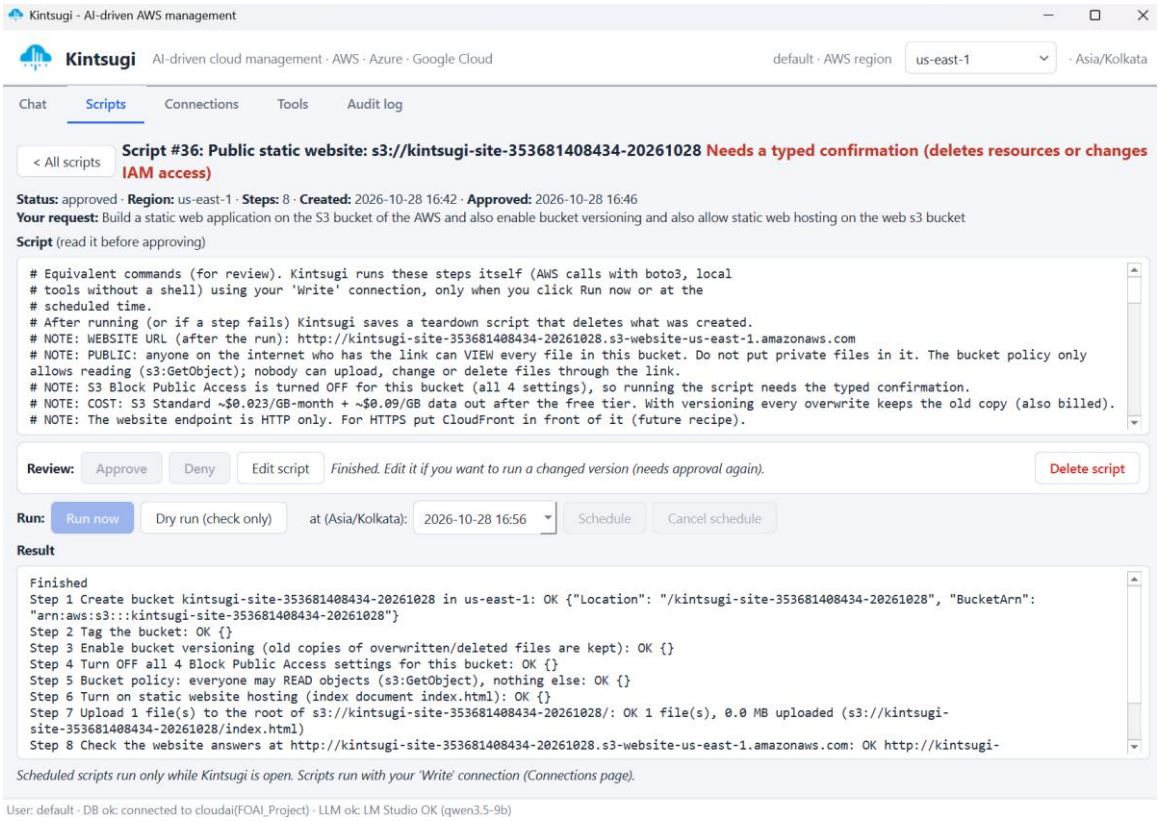


Fig 5.4 Script #36 after approval: all eight steps completed

The result was cross-checked independently in the AWS Management Console. Figure 5.5 shows the new bucket kintsugi-site-353681408434-20261028 in US East (N. Virginia), created at 16:46 IST, and Figure 5.6 shows the uploaded index.html (859 bytes, Standard storage class) with a version ID marked as the current version, which confirms that bucket versioning was enabled as requested.

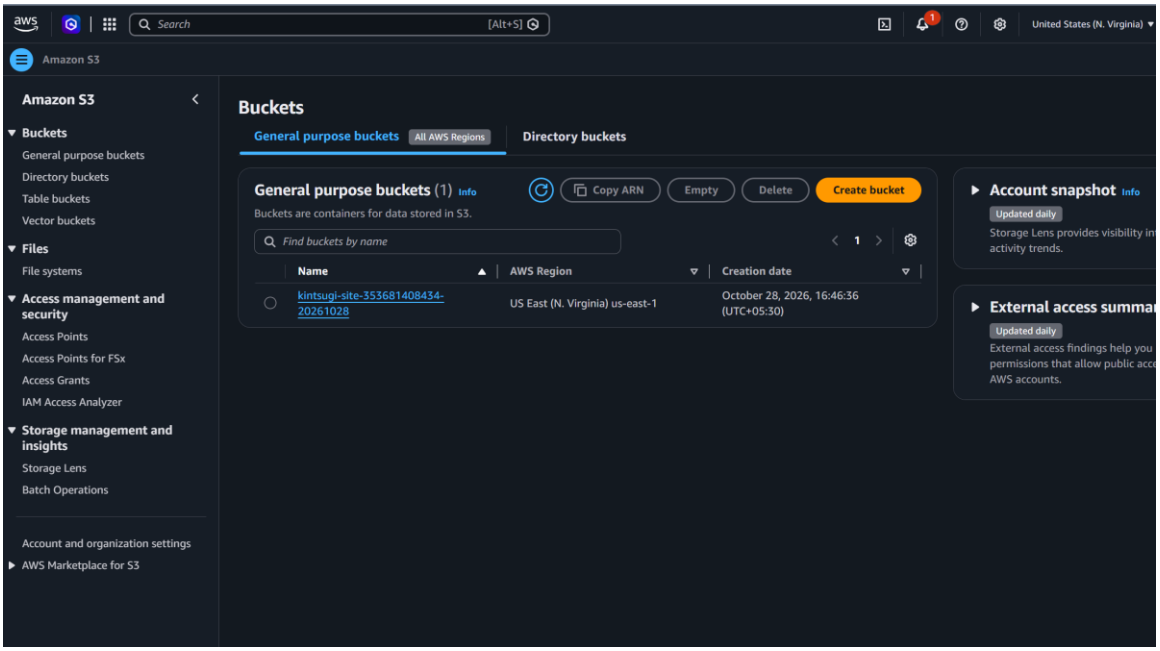


Fig 5.5 AWS console: the bucket created by Kintsugi

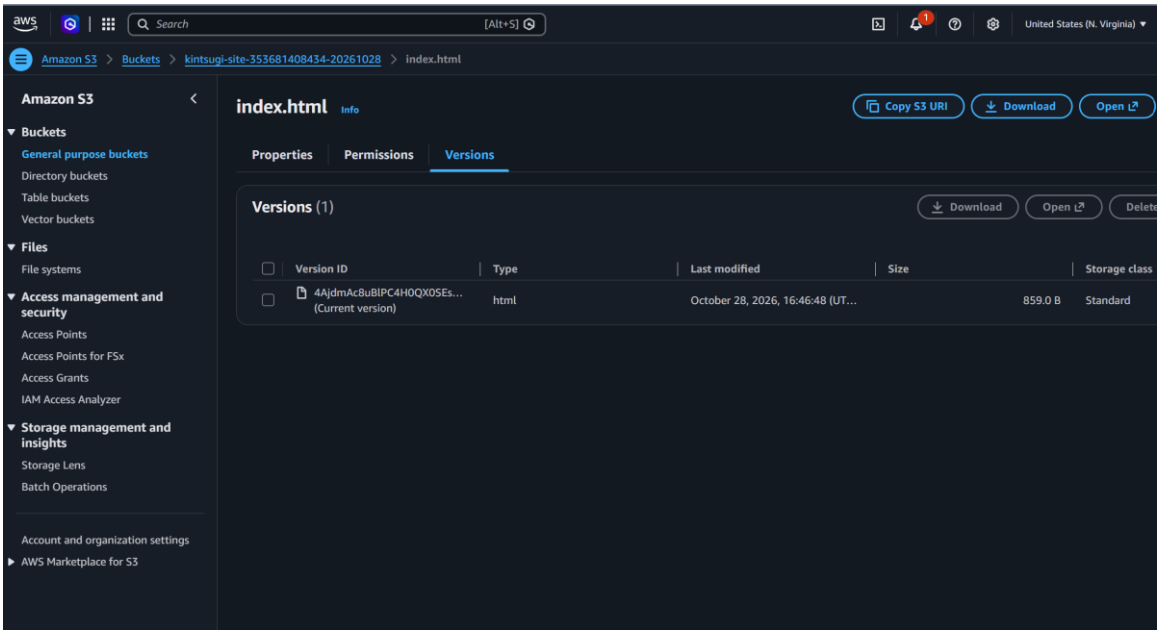


Fig 5.6 AWS console: versioned index.html uploaded to the bucket root

5.3.4 Audit trail

Figure 5.7 shows the Audit log page. Every stage of the run is recorded with time, actor, tool, risk, approval state, status and a result summary: the template call that saved script #36 (pending), the user's approval, each of the eight AWS steps (s3.create_bucket, s3.put_bucket_tagging, s3.put_bucket_versioning, s3.put_public_access_block, s3.put_bucket_policy, s3.put_bucket_website, s3_upload and check:http), all approved and successful. Earlier entries from 14

October are also visible, including failed plan_task calls and a blocked check_prerequisites call, which shows that failures and refusals are logged as faithfully as successes.

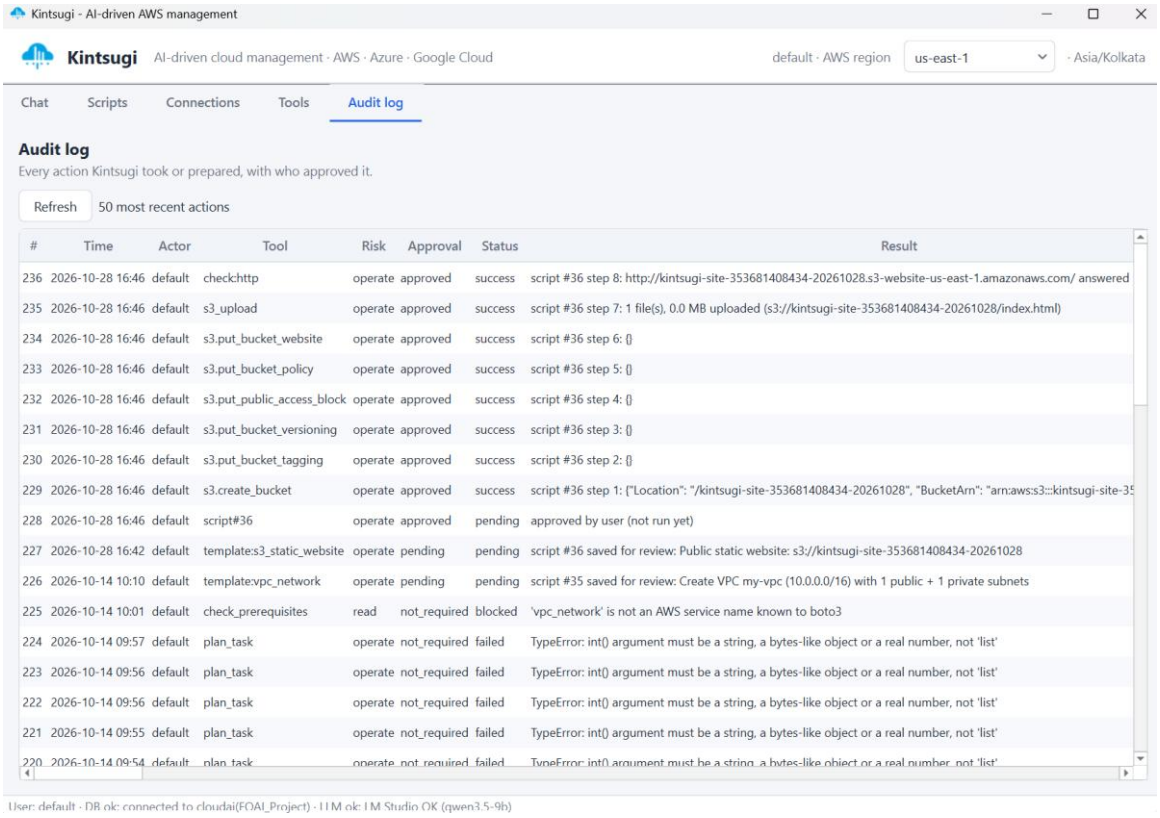


Fig 5.7 Audit log with every step of the run recorded

This case study demonstrates the full chain required by the design: natural-language request, tool selection (with the fallback router when the LLM failed), validation and clarifying questions by code, a reviewable script, explicit human approval with typed confirmation, execution with the write identity, automatic verification and a complete audit record. After the demonstration the generated teardown script can remove the bucket so that no charges continue.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project designed and implemented Kintsugi, an AI-driven desktop application for managing AWS through natural language. The key idea is a strict division of labour: a local language model understands the request and chooses typed tools, while deterministic code calls the cloud, computes every number, enforces security policy and asks the user before anything changes. Supporting AI techniques, namely a rule-based fallback router, frame-based clarifying questions and precondition checking, make the system reliable even with a 14-billion-parameter model running on a laptop. The application covers read access to all AWS services, single write operations and more than twenty-five multi-step recipes for networking, compute, storage, serverless, containers and databases, all behind a human approval workflow, encrypted credential storage and a complete audit log. The project demonstrates that a useful and safe cloud assistant can be built without paid AI services and without giving an AI model unrestricted power. The end-to-end run in Section 5.3, from a chat request to a live public website on Amazon S3 with versioning, confirms that the approval workflow, script engine, verification step and audit log work together against a real AWS account.

6.2 FUTURE WORK

- **Anomaly detection** on CloudWatch metrics using Z-score, moving average and EWMA, followed by Isolation Forest.
- **Cost forecasting** with time-series trend models and per-service cost attribution, explained by the LLM.
- **Recommendation engine** for idle and under-utilised resources with estimated savings.

- **Model comparison** on a benchmark of 15 to 20 prompts, measuring tool-choice accuracy and latency for several local models.
- **Retrieval-augmented generation** over AWS documentation using the nomic embedding model and pgvector.
- **Stronger identity model:** AssumeRole everywhere, per-task IAM grants generated with Terraform, permission boundaries, and secrets in a key management service or the Windows Credential Manager instead of a key file.
- **SSM Session Manager** instead of SSH for server configuration, so no inbound port 22 is needed.
- **Fuller Azure and Google Cloud support**, user accounts with role-based access control, and packaging as a single Windows executable.

CHAPTER 7

REFERENCES

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P.N., Inkpen, K., Teevan, J., Kikin-Gil, R. and Horvitz, E. (2019) 'Guidelines for Human-AI Interaction', Proceedings of the CHI Conference on Human Factors in Computing Systems, pp.1-13.

Brikman, Y. (2022) Terraform: Up and Running, 3rd edn., O'Reilly Media, Sebastopol.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. et al. (2020) 'Language Models are Few-Shot Learners', Advances in Neural Information Processing Systems, Vol.33, pp.1877-1901.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T. and Fritz, M. (2023) 'Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection', Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, pp.79-90.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V. et al. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', Advances in Neural Information Processing Systems, Vol.33, pp.9459-9474.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. et al. (2022) 'Training Language Models to Follow Instructions with Human Feedback', Advances in Neural Information Processing Systems, Vol.35, pp.27730-27744.

Patil, S.G., Zhang, T., Wang, X. and Gonzalez, J.E. (2023) 'Gorilla: Large Language Model Connected with Massive APIs', arXiv preprint arXiv:2305.15334.

Qwen Team (2025) 'Qwen3 Technical Report', arXiv preprint arXiv:2505.09388.

Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach, 4th edn., Pearson, Hoboken.

Saltzer, J.H. and Schroeder, M.D. (1975) 'The Protection of Information in Computer Systems', Proceedings of the IEEE, Vol.63, No.9, pp.1278-1308.

Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N. and Scialom, T. (2023) 'Toolformer: Language Models Can Teach Themselves to Use Tools', Advances in Neural Information Processing Systems, Vol.36.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. and Zhou, D. (2022) 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models', Advances in Neural Information Processing Systems, Vol.35, pp.24824-24837.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023) 'ReAct: Synergizing Reasoning and Acting in Language Models', International Conference on Learning Representations (ICLR).